

Exercícios abertos — Revisão de POO

1. Paradigmas

Explique o que é um paradigma de programação e, em seguida, explique a principal diferença entre a forma de pensar da Programação Funcional e da Programação Orientada a Objetos.

Não responda apenas listando características. Explique **como cada paradigma organiza o desenvolvimento de um programa**.

2. Abstração

Imagine um sistema de uma universidade que possui alunos, professores, disciplinas e turmas.

Explique como o conceito de **abstração** poderia ser utilizado nesse sistema.

Na sua resposta, explique também **qual problema a abstração ajuda a resolver** em sistemas de software.

3. Classe x objeto

Explique a diferença entre **classe e objeto**.

Na resposta, obrigatoriamente explique:

- o que a classe define;
- o que o objeto recebe da classe;
- estado;
- identidade;
- ciclo de vida.

Depois dê um exemplo próprio.

4. Encapsulamento

Imagine que existe uma classe `ContaBancaria` e que o saldo pode ser alterado diretamente por qualquer parte do sistema.

Explique quais problemas isso poderia causar.

Depois explique como o **encapsulamento** poderia resolver esse problema e por que isso ajuda a **reduzir acoplamento e complexidade**.

5. Herança

Explique por que a herança existe na POO.

Imagine as classes:

```
Carro
├── CarroAlcool
├── CarroGasolina
└── CarroFlex
```

Explique o que significa dizer que `CarroAlcool` herda de `Carro`.

Depois explique por que `CarroAlcool` pode ser tratado como um `Carro`, mas `CarroAlcool` não pode ser tratado como um `CarroGasolina`.

6. Referência x objeto

Explique, com suas próprias palavras, o que acontece nesta declaração:

```
Carro c = new CarroAlcool();
```

Sua resposta deve explicar:

- qual é o tipo da referência;
 - qual é o tipo real do objeto;
 - por que essa declaração é permitida;
 - qual relação isso possui com herança;
 - qual relação isso possui com polimorfismo.
-

7. Métodos e referência

Considere:

```
Carro c = new CarroAlcool();
```

Explique por que o fato de o objeto real ser `CarroAlcool` não significa automaticamente que a referência `c` possa acessar qualquer método exclusivo de `CarroAlcool`.

Explique qual é o papel do **tipo da referência** nessa situação.

8. Cast

Explique o que está acontecendo nesta expressão:

```
((CarroAlcool)c).autonomia(10);
```

Por que o cast pode ser necessário nesse caso?

Explique também o que o cast **não faz** com o objeto.

9. Sobrecarga x overriding

Explique a diferença entre **sobrecarga (overloading)** e **sobreposição (overriding)**.

Depois crie um pequeno exemplo conceitual para cada um.

Não basta dizer “um muda parâmetro e o outro muda método”. Explique **por que esses mecanismos existem e qual problema cada um ajuda a resolver**.

10. Polimorfismo

Considere:

```
Polígono  
├── Triângulo  
├── Retângulo  
└── Hexágono
```

Todos possuem:

```
public double calculoArea()
```

Explique como o polimorfismo pode ser utilizado nesse caso.

Depois responda:

Qual seria a vantagem de um sistema trabalhar com **Poligono** em vez de precisar saber antecipadamente se cada objeto é Triângulo, Retângulo ou Hexágono?

11. Classe abstrata

Explique a frase:

“Podemos criar variáveis do tipo de uma classe abstrata, porém não podemos criar instâncias de uma classe abstrata.”

Depois explique a diferença entre:

Carro c;

e

Carro c = new Carro();

considerando que **Carro** seja abstrata.

12. Questão integradora

Imagine que você está desenvolvendo um sistema para uma locadora de veículos.

Existem diferentes tipos de veículos, cada um com características e comportamentos próprios.

Explique como você utilizaria, nesse sistema:

abstração → classes → objetos → encapsulamento → herança → overriding → polimorfismo

Não precisa escrever código. O objetivo é mostrar **como um conceito leva ao outro e por que cada um seria útil no sistema.**

13. Questão estilo professor

Explique por que podemos dizer que **POO está relacionada à tentativa de controlar a complexidade dos sistemas de software.**

Na resposta, relacione pelo menos **abstração, encapsulamento e polimorfismo.**

14. Questão de raciocínio

Um aluno afirma:

“Se **Carro c = new CarroAlcool();**, então **c** é um objeto Carro e não um objeto CarroAlcool.”

Você concorda ou discorda?

Explique detalhadamente o que está errado ou correto nessa afirmação, diferenciando **variável/referência** e **objeto**.

15. A mais importante

Explique, como se estivesse respondendo ao professor:

Por que a POO possui conceitos como abstração, encapsulamento, herança e polimorfismo?

Não defina cada conceito separadamente. Tente construir uma explicação mostrando **qual problema geral eles tentam resolver e como cada mecanismo contribui para organizar um sistema de software**.